

Sakarya University, Department of Software Engineering
SWE 204 Concepts of Programming Languages, Spring 2026
Homework 2

Groups: 2-4 people

Submission: Code + Report (21 May 2026, Thursday, 09:00)

In-class demo: ~5 minute per group (21 May 2026, Thursday, 09:00 – 13:00)

Demo schedule: Please reserve a slot for your group:

https://docs.google.com/spreadsheets/d/1TaKP_Hw7Ep3Yz_DzQMPbgFaKzA53j7WpBWMqkrebe9Q/edit?usp=sharing

In this homework, you are expected to apply concepts of subprograms, exception handling, and performance benchmarking through the implementation of a matrix multiplication task. You will complete the assignment in three different languages, preferably C++, Java, and Python.

1. Matrices

Your program must be able to read two matrices from a single text file. Several input files are provided containing matrix pairs of different sizes, ranging from small matrices (e.g., 50×80 and 80×60) to very large matrices (e.g., up to 900×700 and 700×1000). Some input files intentionally contain errors to test your exception handling implementation.

<pre>n m (matrix A) p q (matrix B)</pre>

Each input file contains exactly two matrices separated by a blank line, and your program must correctly read both matrices according to the specified format.

The first line of each matrix specifies its dimensions, where Matrix A has size $n \times m$ and Matrix B has size $p \times q$. Each subsequent line represents a row of the matrix and contains integer values between 0 and 9, separated by spaces.

After reading both matrices, your program should validate that the matrices are compatible for multiplication ($m = p$) and then perform matrix multiplication to produce the resulting matrix.

During the demo session, your program will be required to read matrices from an input file. However, for benchmarking experiments, you may directly use the matrix generator code provided in the homework package instead of reading from files.

2. Exception handling

Your program must detect and handle possible input errors during matrix reading and validation. The following error types must be checked:

- Missing elements in a row
- Extra elements in a row
- Non-numeric values
- Incompatible matrix dimensions for multiplication ($m \neq p$)

When an error is detected, your program must print an appropriate error message and stop execution for that file.

3. Benchmarking

In this part, you will observe how execution time changes as matrix size increases and how different data structures affect memory allocation and memory usage behavior across different programming languages.

Benchmark 1: Data Structure Comparison (Memory Allocation Focus)

The goal is to observe how memory allocation strategies and internal memory representations affect performance when handling large matrices.

For each language, implement the same matrix operations using the following data structures:

- C++:
 - dynamic array (new/delete)
 - vector (vector<vector<int>>)
- Java:
 - array (int[][])
 - ArrayList (ArrayList<ArrayList<Integer>>)
- Python:
 - list of lists

Benchmark 2: Direct Computation vs. Function Call

The goal is to observe the effect of function calls on execution time. The same computation must be implemented in two ways:

- **Direct computation:** Perform multiplication directly inside the loop.

$C[i][j] = A[i][j] * B[i][j];$

- **Function-based computation:** Call a function for multiplication.

`C[i][j] = multiply(A[i][j], B[i][j]);`

In this benchmark , the same data structure must be used in both versions so that only the effect of function calls is measured.

Reporting results:

For all experiments, use increasing matrix sizes such as 50×80, 120×150, 300×400, 900×700, and so on. If needed to better observe performance differences, you may adjust the matrix sizes (increase or decrease them) or choose alternative sizes.

For each configuration, run the program 5 times and compute the average execution time. Record results in milliseconds (ms).

Overall, you are expected to observe and research how memory allocation strategies influence performance across implementation choices and different programming languages.

Record results in a table. Additionally, you may generate a line chart showing how execution time changes as matrix size increases.

Example table format:

Language	Implementation	Size	Avg. Time (ms)
C++	Dynamic array	50×80	...
C++	Dynamic array	120×150	...
...			
C++	Vector	50×80	...
C++	Vector	120×150	...
...			
Java	Direct computation	...	...
Java	Function Call	...	...
...			
Python	Direct computation	...	...
Python	Function Call	...	...
...			